

	Break it into vertical stories				
	Story	Client	Gateway	QA	Shippable?
	PLA - Build Messanging API	✓	✓	✓	
	PLA -	✓	✓	✓	

		Trunk Based development	Ensures at no given point we've breaking changes in branch.		
		At clients we can have Sprint bound branches for every feature, should be raised against release.	Ensure to check all release checkboxes		
		Everday afternoon checkpoint, review call for PRs, so we can merge them EOD in sprint trunk.			
		Devs local branches that last under 24 hours			
		From Sprint trunk we have to follow strict timelines: Cut_Off - No changes will be entertained, Hardening- No new feature addition, Sprint_Release-merge into release branch timelines			
	Feature Flag				
		We'll use local switches which are off, as not every feature is ready in 2 weeks some might take 3 week.	Local config files, compiletime safety of work not being executed, no server new swtch is required	This is save us from breaking release branch and live customer builds.	Flags.swift, Flags.kt, or flags.ts
	CI/CD Setup				
		With a 2-week cycle, manual testing will crush your momentum. You need your pipeline to do the heavy lifting	Linting, Unit testing should run on builds before merge.		
	1	Optimize Bitrise for < 10 Minute PRs			
		With 4 developers merging code daily, pipeline bottlenecks will kill momentum. Separate your feedback loops inside Bitrise:			
		The Pull Request Workflow (Fast Gate):			
		Run spotlessCheck or detekt for code style.			
		Run only unit tests: ./gradlew testDebugUnitTest.			
		Skip Android UI tests (connectedCheck), Emulators, and heavy obfuscation (Minify/R8).			
		Goal: Keep this under 5 to 7 minutes. [1, 2]			
		The Nightly Main Workflow (Deep Verification):			
		Trigger automatically every night at midnight from the main branch.			
		Run ./gradlew assembleDebug and push an APK/AAB to Firebase App Distribution or Google Play Internal Testing .			
		This ensures the team always has a fresh, testable build of the trunk every morning. [1]			
	2	Implementation: Dead-Simple Kotlin Feature Flags	Flags.swift, Flags.kt, or flags.ts		

	3	Gradle Modularization (The TBD Enabler)	Already in place		
	4	The 2-Week Branching Automation in Bitrise			
		Map your 2-week "Release Train" directly to Bitrise triggers:			
		[Day 1-10: Main Branch] ----> Bitrise Nightly Build ----> Internal Testers (QA)			
		[Day 11: Cut Release] -----> Create branch `release/v2.1.0`			
		[Day 11-14: Hardening] -----> Bitrise Release Workflow ----> Google Play Beta			
		[Day 14: Store Launch] -----> Tag `v2.1.0` ----> Production Store			
		The Release Workflow: Configure Bitrise to trigger a production-ready build (./gradlew bundleRelease) only when a branch matching release/* is created or a Git tag is pushed.			

		We've to answer few questions before we start jumping into coding and we've to record them in given template below.	Template
		For High level there will be team level call with Senior SAs there to support us, teams notes will be stored for ref. No direct contact for individual exploration no confusion, check with your team mates.	Regular interval calls set up.

		Definition of Done	
		Every story must satisfy:	
		Area	Done?
		Client implemented	
		Gateway/API implemented	
		Backend implemented	
		Unit tests	
		Integration tests	
		QA passed	
		Feature flag (if needed)	
		Monitoring/logging	
		Deployable	

	Before adding a story to a sprint, ask:		
	Can a customer use this if we release it in two weeks?		
	Does it include all required layers (client, gateway, backend, testing)?		
	Can it be demonstrated end-to-end without saying "the backend isn't ready yet" or "the UI will come next sprint"?		
	Can it be hidden behind a feature flag if it isn't ready for all users?		
	Three categories of work		
	I usually classify backlog items like this:		
	Type	Customer visible?	Should ship?
	Feature	Yes	Yes
	Platform/Infrastructure	No	Yes
	Technical debt	No	Sometimes
	The important thing is that platform work should still have a clear outcome.		
	A decision framework I recommend		
	For each gateway-heavy story, ask these questions:		

	Can this be deployed to production independently?		
	Does it provide a reusable capability (API, resilience, observability, security, performance)?		
	Can another team start using it immediately, even if no UI changes are released yet?		
	Does it reduce the lead time for future customer features?		
	If the answer is "yes" to most of these, it is a healthy gateway story.		
	Backlogs		
	For organizations with distinct Client, Gateway, and QA teams, one practice used by companies like Netflix and Spotify is to maintain two synchronized backlogs:		
	Customer value backlog: end-to-end features delivered as vertical slices.		
	Platform enablement backlog: APIs, security, resilience, performance, migrations, and infrastructure that unblock or accelerate future features.		